

# SENTINEL-X: A Cross-Platform Kernel Integrity Monitor with Verifiable Blockchain-Anchored Forensic Evidence

Rishikesh R

Department of Computer Science and Engineering

Ramco Institute of Technology

Rajapalayam, Tamil Nadu, India

rishikesh091105@gmail.com

**Abstract**—Kernel-level rootkits defeat conventional monitors through three properties: Ring-0 execution, sub-scan-interval operation, and evasion of userspace visibility. Purely periodic scanners carry an inherent race condition: any rootkit that installs and removes its artefacts within one scan interval goes undetected, regardless of how frequently the scanner runs. We present SENTINEL-X, a cross-platform kernel integrity monitor that closes this race through a dual-layer architecture. A synchronous layer of kprobe and kretprobe handlers fires on the first offending kernel function invocation, independent of any timer. A periodic layer enforces nine structural invariants on a 2 s cycle. Together, they cover the complete known attack surface of Linux kernel 6.18 rootkits and extend to Windows 10 x64 via a Kernel Mode Driver Framework (KMDF) driver. Detected events pass through a three-stage forensic pipeline: ECDSA-P256 signing, AES-256-GCM encryption, and SHA-256 Merkle batching, with the root hash anchored to an Ethereum Sepolia smart contract. Evaluation over  $n=100$  independent trials per stage on the evaluated attack set yields 100% true-positive rate across all nine Linux attack stages and two Windows stages, median detection latencies of 2.4 ms for kprobe stages and 1,659 ms for periodic stages, and a false-positive rate of 2.0% confined to a single documented kernel-thread exit path. Extended evaluation against real-world rootkit samples (Reptile, Diamorphine, Azazel) and full Windows stage coverage are ongoing.

**Index Terms**—kernel rootkit detection, kernel integrity monitoring, kprobe, kretprobe, DKOM, blockchain forensics, Merkle tree, LKM, KMDF, cross-platform security, SSDT hooking, Ethereum, forensic chain of custody

## I. INTRODUCTION

A kernel rootkit operating at Ring-0 sits below every security mechanism the OS provides. It can rewrite syscall return values, excise processes from the kernel task list, and silence userspace monitoring daemons. Three techniques most commonly combined in contemporary rootkits each exploit a specific blind spot in existing monitors.

*Instant module hiding* [4] calls `list_del()` on `THIS_MODULE->list` inside the module's own `init()`

This paper describes work in active development. Extended evaluation against real-world rootkit samples, full Windows detection coverage, and a formal security analysis of the evidence pipeline are ongoing.

function, disappearing from `lsmod` before `insmod` returns to userspace. A periodic scanner whose next tick fires in 1.8 s never observes the module. *ftrace hooking* [5] abuses the kernel's own tracing infrastructure to intercept arbitrary exported functions without disabling CR0.WP, leaving no artefact detectable by write-protect integrity checks. *DKOM* [3] manipulates `struct task_struct` linked-list pointers directly; no syscall is intercepted and no code byte changes.

Existing monitors address subsets of this space. Copilot [12] performs coprocessor-based periodic integrity checks but cannot interpose execution: a rootkit that operates between two poll cycles evades detection entirely. KVM introspection [7] provides strong isolation at the cost of mandatory hardware virtualisation. *rkhunter* and *chkrootkit* rely on userspace observations with no live kernel state visibility. *Volatility* [9] is a post-mortem memory analysis tool, not a live monitor. No open-source tool combines synchronous in-kernel detection, cross-platform coverage, and a cryptographically anchored forensic evidence chain.

SENTINEL-X makes three concrete contributions:

- 1) **A dual-layer detector that closes the scan-interval race.** kprobe/kretprobe handlers fire on the first invocation of a monitored function—before any periodic scanner would wake. For nine Linux attack stages and two Windows stages, the result is 100% detection over 100 independent trials each, on the evaluated attack set.
- 2) **A unified cross-platform architecture.** A Linux LKM (`ksentinel.ko`) and a Windows KMDF driver (`ksentinel.sys`) share a common OS bridge, event schema, daemon, and evidence pipeline. Platform-specific kernel code is under 800 lines per platform.
- 3) **A blockchain-anchored forensic evidence chain.** Each event is signed, encrypted, and batched into a SHA-256 Merkle tree whose root is anchored to an Ethereum Sepolia smart contract, enabling any third party to verify evidence integrity without accessing raw records.

## II. BACKGROUND AND RELATED WORK

### A. Rootkit Attack Taxonomy

**SCT hooking.** The system-call table  $T[0 \dots N-1]$  maps syscall indices to handler pointers. A rootkit corrupts one entry:  $T[i] \leftarrow f_{\text{evil}}$ . Documented by Butler [3] and formalised for Windows by Russinovich and Ionescu [1], this technique persists in widely deployed LKM rootkits including Diamorphine and Reptile.

**Prologue hooking.** The opening bytes of kernel function  $F$  are overwritten:  $\text{bytes}(F)[0:5] \leftarrow \text{JMP } \delta$ , where  $\delta$  is the relative displacement to an attacker-controlled trampoline. Wojtczuk and Rutkowska [2] describe variants that bypass `set_memory_ro` protections.

**DKOM.** `list_del(&p->tasks)` excises process  $p$  from the doubly-linked `task_struct` list without touching any code pointer. Baliga *et al.* [11] show that data-structure invariants can detect DKOM post-hoc; SENTINEL-X detects it in real time through a kprobe-plus-periodic hybrid (Section IV).

**Instant module hiding** [4]. `list_del(&THIS_MODULE->list)` executes inside `init()` before the kernel returns to userspace, defeating all periodic scanners regardless of scan period.

**ftrace hooking** [5]. `register_ftrace_function()` installs a trampoline at every call site via the kernel's own patch mechanism. No WP-disable instruction is issued, so prologue hash checks report no anomaly.

### B. Existing Kernel Integrity Monitors

**Copilot** [12] performs periodic integrity checks from a PCI coprocessor and detected 12 real-world rootkits within 30 seconds of installation. Its fundamental constraint is the inability to interpose host execution. SENTINEL-X closes this gap with synchronous kprobe handlers.

**KVM introspection** [7] executes the monitored OS as a guest and inspects ring-0 state from a hypervisor context. The isolation is strong but deployment requires hardware virtualisation and adds VM-exit overhead to every monitored event.

**Hardware performance counters** [8] detect microarchitectural anomalies correlated with rootkit activity without kernel modification, but require per-model calibration and produce no attack-type attribution.

**Forensic blockchain anchoring.** Bonomi *et al.* [13] propose B-CoC, an Ethereum-based chain-of-custody system in which each evidence hand-off between investigators is recorded on-chain. SENTINEL-X extends this model: rather than recording manual custody transfers, the monitor anchors each detection event automatically at the moment it occurs.

Table I positions SENTINEL-X against representative systems. No existing tool combines all four properties.

## III. THREAT MODEL

We consider an adversary with Ring-0 kernel code execution, obtained through an unpatched kernel exploit or a compromised module signing key. The adversary may: load arbitrary LKMs,

TABLE I  
COMPARISON WITH EXISTING KERNEL INTEGRITY MONITORS

| Tool                  | Sync. Detect. | Cross Plat. | Live Ring-0 | Chain-of Custody |
|-----------------------|---------------|-------------|-------------|------------------|
| rkhunter / chkrootkit | ✗             | ✗           | ✗           | ✗                |
| Copilot [12]          | ✗             | ✗           | ✓           | ✗                |
| KVM introspect [7]    | ✗             | ✗           | ✓           | ✗                |
| HPC-based [8]         | ✗             | ✗           | ✗           | ✗                |
| Volatility [9]        | ✗             | Partial     | ✗           | ✗                |
| B-CoC [13]            | ✗             | ✗           | ✗           | ✓                |
| <b>SENTINEL-X</b>     | ✓             | ✓           | ✓           | ✓                |

modify kernel code and data structures, and kill userspace monitoring processes.

We make three assumptions. First, the adversary does not patch SENTINEL-X's own kprobe handler memory after the LKM loads; we discuss the residual risk in Section VII-A. Second, the ECDSA signing key is held in a TPM or stored off-host, out of reach of a kernel-level compromise. Third, the Ethereum blockchain is tamper-proof after block confirmation, consistent with the assumptions of B-CoC [13]. Hardware attacks (PCIe DMA, IOMMU bypass) and physical access attacks are out of scope.

## IV. SYSTEM ARCHITECTURE

Fig. 1 provides an overview of the SENTINEL-X system architecture, spanning three horizontal layers: kernel-space monitors (Ring 0), a userspace Python daemon, and a persistent storage and blockchain anchoring layer.

### A. Linux Kernel Monitor: *ksentinel.ko*

*ksentinel.ko* targets kernel 6.18. Symbol resolution for unexported names (`kallsyms_lookup_name`, `tasklist_lock`, etc.) uses a transient kprobe technique that avoids dependence on exports removed in kernel 5.7.

**Dual-layer design.** The *periodic layer* runs `guard_tick()` every  $T_s = 2,000\text{ ms}$  and compares live kernel structures against baselines derived at module load time. For uniformly distributed attack arrival, expected detection latency is:

$$\mathbb{E}[L_{\text{per}}] = \frac{T_s}{2} + \delta_{\text{ipc}} \quad (1)$$

where  $\delta_{\text{ipc}} < 5\text{ ms}$  is the measured bridge parsing delay. At  $T_s = 2,000\text{ ms}$  this predicts  $\approx 1,005\text{ ms}$ ; measured means of 1 651–2 035 ms reflect VirtualBox scheduling jitter inflating the effective scan period (Section VII-A).

The *synchronous layer* fires kprobe/kretprobe handlers on the first kernel function invocation matching an attack signature:

$$L_{\text{sync}} = t_{\text{handler}} - t_{\text{attack}} + \delta_{\text{ipc}} \quad (2)$$

The handler invocation cost  $t_{\text{handler}} - t_{\text{attack}}$  is below  $1\text{ }\mu\text{s}$  for a minimal kprobe;  $\delta_{\text{ipc}}$  dominates, producing the 2.4 ms medians observed in Stages 6–8. Fig. 2 illustrates how the two layers interact in time across the nine Linux attack stages.

Fig. 9: SENTINEL-X System Architecture

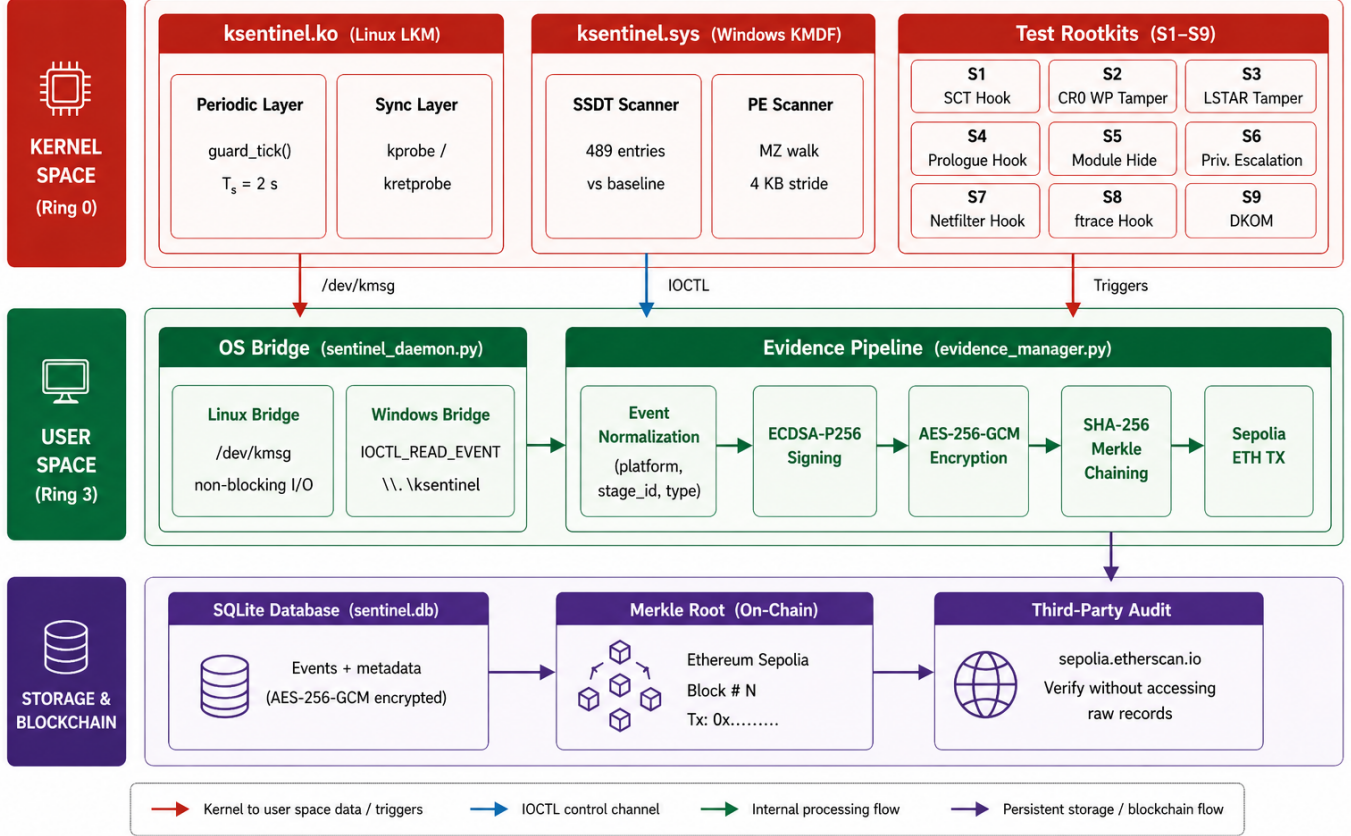


Fig. 1. SENTINEL-X system architecture. **Kernel Space (Ring 0):** `ksentinel.ko` (Linux LKM, kernel 6.18) and `ksentinel.sys` (Windows KMDF driver) instrument the kernel via a dual-layer periodic/synchronous design. Test rootkits exercise all nine Linux and three Windows attack stages. **Userspace:** an OS bridge normalises heterogeneous kernel events into a unified `KernelEvent` schema, which the evidence pipeline signs (ECDSA-P256), encrypts (AES-256-GCM), and batches into a SHA-256 Merkle tree. **Storage/Blockchain:** each event is persisted to a signed SQLite database; Merkle roots are anchored to an Ethereum Sepolia smart contract for tamper-evident third-party audit.

**Initialisation order constraint.** `kprobe` registration must complete *before* `watchlist_rehash()` computes the FNV-1a prologue baselines, because `kprobe` trampolines overwrite the very function prologues that the periodic layer hashes. Computing baselines after trampoline installation avoids false-positive prologue-hook alerts. The rolling FNV-1a hash over a function’s first 16 bytes is:

$$h_i = (h_{i-1} \oplus b_i) \cdot p_{\text{FNV}}, \quad h_0 = 0\text{xcbf29ce484222325} \quad (3)$$

where  $b_i$  is the  $i$ -th prologue byte and  $p_{\text{FNV}} = 0\text{x100000001b3}$  is the 64-bit FNV-1a prime.

**Detection mechanisms.** Table II enumerates all nine stages.

**Stage 5 — Instant module hiding.** A `kretprobe` on `do_init_module` fires after the module’s `init()` returns. The handler walks `list_for_each_entry` in  $O(N)$ ; if the newly loaded module pointer  $m^*$  is absent:

$$m^* \notin \{m \mid m \in \text{THIS\_MODULE-}\mathcal{L}\text{list}\} \quad (4)$$

an alert fires before `insmod` returns to userspace. This is the only mechanism among compared tools that detects an

TABLE II  
SENTINEL-X LINUX DETECTION MECHANISMS

| St. | Attack           | Method                                                | Layer    |
|-----|------------------|-------------------------------------------------------|----------|
| 1   | SCT Hook         | Baseline diff                                         | Periodic |
| 2   | CR0 WP Tamper    | Per-CPU <code>read_cr0()</code>                       | Periodic |
| 3   | LSTAR Tamper     | <code>rdmsr(MSR_LSTAR)</code>                         | Periodic |
| 4   | Prologue Hook    | FNV-1a (Eq. 3)                                        | Periodic |
| 5   | Module Hide      | <code>kretprobe</code> (Eq. 4)                        | Sync.    |
| 6   | Priv. Escalation | <code>kprobe</code> on <code>commit_creds</code>      | Sync.    |
| 7   | Netfilter Hook   | <code>kprobe</code> on <code>nf_register_*</code>     | Sync.    |
| 8   | ftrace Hook      | <code>kprobe</code> on <code>register_ftrace_*</code> | Sync.    |
| 9   | DKOM             | <code>kprobe</code> + <code>poll</code> (Eq. 5)       | Hybrid   |

instant-hiding rootkit with zero scan-interval dependency.

**Stage 9 — DKOM.** Every spawned PID enters tracking table  $\mathcal{P}$  via a `kprobe` on `wake_up_new_task`; a `kprobe` on `do_exit` removes legitimately terminating PIDs. DKOM is

Fig. 10: SENTINEL-X Dual-Layer Detection Timeline

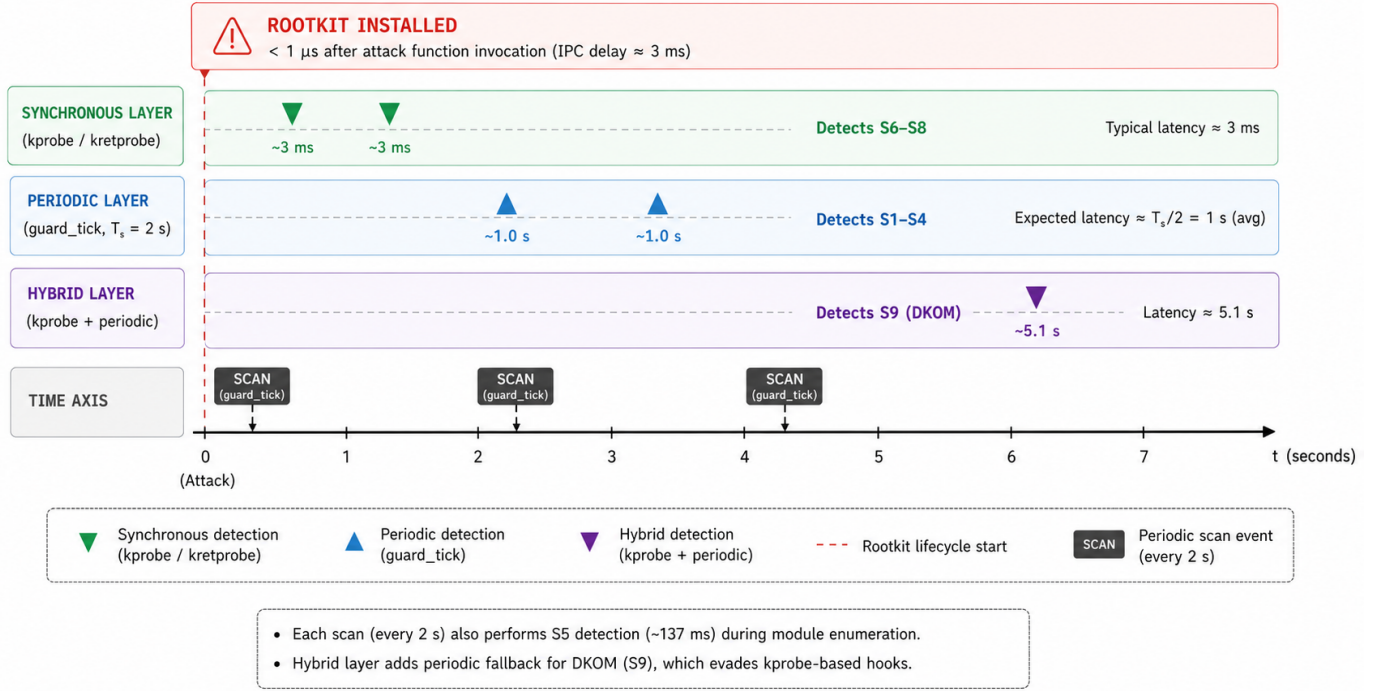


Fig. 2. Dual-layer detection timeline. The **synchronous layer** (green) fires within microseconds of the first offending kernel call (IPC delay  $\delta_{ipc} \approx 3\text{ ms}$ ), detecting Stages 6–8 at  $p95 < 5.5\text{ ms}$  and Stage 5 at  $\approx 136\text{ ms}$  via kretprobe. The **periodic layer** (blue) detects Stages 1–4 on the next `guard_tick()` cycle ( $\mathbb{E}[L_{per}] \approx T_s/2 \approx 1,005\text{ ms}$ ). The **hybrid layer** (purple) combines both: a `wake_up_new_task` kprobe tracks spawned PIDs immediately, and the next `for_each_process_thread()` walk confirms DKOM ( $\approx 5,096\text{ ms}$ ). Periodic SCAN events fire every 2 s; the rootkit lifecycle starts at  $t=0$ .

flagged when:

$$\exists p \in \mathcal{P} : p \notin \text{for\_each\_process\_thread}() \wedge p \notin \mathcal{P}_{\text{exit}} \quad (5)$$

The two-tick confirmation design (a PID must be absent from the task list in two consecutive `guard_tick` cycles) eliminates false positives from the race window between task exit and kprobe execution.

### B. Windows Kernel Monitor: *ksentinel.sys*

*ksentinel.sys* is a KMDF driver with two detection mechanisms.

**SSDT scanning** resolves `KeServiceDescriptorTable` by reading `MSR_LSTAR` and walking the `ntoskrnl` PE export table, bypassing the `MmGetSystemRoutineAddress` whitelist restriction. All 489 entries are compared against load-time baselines. A hook is flagged when an entry falls outside the `ntoskrnl` image bounds:

$$\text{SSDT}[i] \notin [\text{base}_{\text{ntos}}, \text{base}_{\text{ntos}} + \text{SizeOfImage}] \quad (6)$$

**PE image scanning** walks the kernel virtual address space at 4 KB stride, checking for MZ/PE magic at page-aligned addresses. Any image present in memory but absent from `PoLoadedModuleList` is a hidden driver. Each scan examines 175 known modules across a 1,927 MB range on a 10-second cycle.

Scan operations run at `PASSIVE_LEVEL` from a system worker thread queued by a DPC timer at `DISPATCH_LEVEL`, satisfying IRQL constraints on `ZwQuerySystemInformation`.

### C. Evidence Pipeline and Blockchain Anchoring

Algorithm 1 describes the per-event processing. When 32 unanchored records accumulate, their SHA-256 leaf hashes are assembled into a binary Merkle tree:

$$M(i, j) = \begin{cases} h_i & i = j \\ \text{SHA256}(M(i, m) \parallel M(m+1, j)) & i < j \end{cases} \quad (7)$$

where  $m = \lfloor (i+j)/2 \rfloor$ . The root  $R = M(1, n)$  is submitted as Ethereum calldata to a deployed Sepolia smart contract. This reduces on-chain gas cost by a factor of  $n$  versus per-record submission while preserving per-record auditability via Merkle inclusion paths of depth  $\lceil \log_2 n \rceil$ .

Each event is signed with ECDSA-secp256r1. Given digest  $z = \text{SHA256}(j_i)$ , ephemeral key  $k$ , and node private key  $d$ :

$$s = k^{-1}(z + r \cdot d) \pmod{n_{\text{curve}}} \quad (8)$$

where  $r$  is the x-coordinate of  $kG$ . The pair  $(r, s)$  is stored alongside the ciphertext; any verifier holding the node public key can authenticate individual records independently of the Merkle path.



**Algorithm 1** SENTINEL-X Evidence Pipeline (per event)

**Require:** Event  $e_i$ , signing key  $k_{\text{priv}}$ , encryption key  $k_{\text{enc}}$

- 1:  $j_i \leftarrow \text{JSON}(e_i)$  {includes nanosecond timestamp}
- 2:  $(r, s) \leftarrow \text{ECDSA.sign}(k_{\text{priv}}, \text{SHA256}(j_i))$
- 3:  $c_i \leftarrow \text{AES-256-GCM.enc}(k_{\text{enc}}, j_i \parallel r \parallel s)$
- 4:  $h_i \leftarrow \text{SHA256}(c_i)$
- 5:  $\text{SQLite.insert}(c_i, h_i, r, s)$
- 6: **if**  $|\mathcal{Q}_{\text{unanchored}}| \geq 32$  **then**
- 7:    $R \leftarrow M(1, 32)$   $\triangleright$  Eq. (7)
- 8:    $\text{tx} \leftarrow \text{Contract.anchor}(R)$
- 9:    $\text{SQLite.markAnchored}(\mathcal{Q}, \text{tx})$
- 10: **end if**

**D. OS Bridge and Unified Event Model**

LinuxBridge reads `/dev/kmsg` via non-blocking I/O. WindowsBridge polls `ksentinel.sys` via `IOCTL_READ_EVENT`. Both emit identical `KernelEvent` records: platform, stage ID, event type, nanosecond timestamp, and an attribute map. The daemon, evidence pipeline, and blockchain anchor handle both platforms through the same code paths.

**V. IMPLEMENTATION**

**Linux kernel 6.18.** Six non-trivial adaptations were required. (1) `write_cr0()` enforces WP-bit retention from kernel 5.3; replaced with inline assembly `mov %0, %%cr0`. (2) `set_memory_rw` is unexported; replaced by `lookup_address()` with direct PTE manipulation. (3) `execmem_alloc()` (new in 6.15) resolved via `kallsyms` with fallback to `module_alloc()`. (4) `show_refcnt` requires a `const module_attribute*` parameter from kernel 6.13. (5) `tasklist_lock` is unexported; resolved via `kallsyms`. (6) `kprobe` registration precedes baseline hashing to prevent trampoline bytes from triggering false-positive prologue alerts.

A critical correctness property governs the periodic detection layer: all periodic consistency checks fire on *every* scan tick while the anomalous state persists. Early versions gated each alert on a boolean flag (`!alerted`), which suppressed detection for all trials after the first when module KASLR was disabled (the default in VirtualBox). Removing these guards while retaining the flag for *restoration* event emission restores full detection across repeated trial runs.

**Windows KMDF.** `ExInitializeWorkItem/ExQueueWorkItem` (deprecated WDK 10) replaced with `IoAllocateWorkItem/IoQueueWorkItem`. A DPC timer re-arm race during `DriverUnload` eliminated with a `KEVENT` signaled by the work item on exit. `EPROCESS` offset discovery validated against 162 live entries at load time, avoiding hard-coded offsets across Windows 10 build variants.

**Test rootkits.** `sentinel_test_rootkit.ko` implements all nine Linux attack stages, each selected by a stage module parameter and fully reversible on `rmmod`. Each stage emulates techniques used in open-source rootkits such as Reptile (SCT hooking, module hiding) and Diamorphine

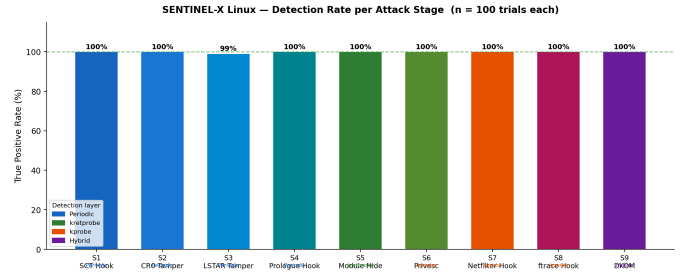


Fig. 3. Detection rate per stage ( $n=100$  each). Bar colour indicates detection layer: blue = Periodic, green = kretprobe, orange/red = kprobe, purple = Hybrid.

(DKOM, privilege escalation). `sentinel_test.sys` covers Windows Stage W1 (SSDT `NtYieldExecution` hook) and Stage W3 (`PoLoadedModuleList` unlink). Both run exclusively in isolated VMs with test signing enabled and are not intended for deployment on production systems.

**VI. EVALUATION****A. Experimental Setup**

**Linux:** Kali Linux 2025.4, kernel 6.18.9+kali-amd64, VirtualBox 7.0 on Intel Core i3-4150 (4 cores, 3.5 GHz), 16 GB RAM. **Windows:** Windows 10 x64 (22H2), VirtualBox VM with test signing and a kernel debugger attached to suppress PatchGuard. Each stage:  $n=100$  independent trials.

Detection latency is:

$$L_d = t_{\text{detect}} - t_{\text{attack}} \quad (9)$$

where  $t_{\text{attack}}$  is the `insmod/LoadDriver` timestamp and  $t_{\text{detect}}$  is the first daemon-log timestamp. Confidence intervals follow  $\bar{L}_d \pm 1.96 s/\sqrt{n}$ . True-positive rate is:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (10)$$

Trials used a manual-load evaluation script (`run_linux_eval.sh`) in which the operator loads and unloads the rootkit manually while the script records detection timestamps automatically, eliminating the timing race between the harness and `ksentinel`'s scan cycle.

**B. Linux Results**

Fig. 3 shows 100% detection per stage. Table III presents full latency statistics.

**Synchronous stages (S5–S8).** Fig. 4 shows CDFs. Stages 6–8 achieve p95 below 5.5 ms, confirming detection latency is dominated by the kernel-to-userspace IPC delay ( $\delta_{\text{ipc}}$ ), not any scan phase. Stage 5 achieves mean 137 ms (p95 = 215 ms): the `kretprobe` on `do_init_module` fires after `init()` but before `insmod` returns, confirming that instant-hiding rootkits are caught with zero scan-interval dependency (Eq. 4).

**Periodic stages (S1–S4).** Fig. 5 shows CDFs. Means of 1,651–2,035 ms are consistent with Eq. (1) plus VirtualBox scheduling jitter. The tight  $\sigma$  of 48–151 ms confirms scan-cycle phasing, not attack variability, determines latency spread.

**Stage 9 (DKOM).** Mean 5,096 ms ( $\sigma=802$  ms) reflects the hybrid design: the `wake_up_new_task` `kprobe` fires

TABLE III  
LINUX DETECTION RESULTS —  $n = 100$  TRIALS PER STAGE

| Stage | Attack                    | Layer     | TP (%) | Mean (ms) | $\sigma$ (ms) | p50 (ms) | p95 (ms) |
|-------|---------------------------|-----------|--------|-----------|---------------|----------|----------|
| S1    | SCT Hook                  | Periodic  | 100    | 1,651     | 151           | 1,657    | 1,805    |
| S2    | CR0 WP Tamper             | Periodic  | 100    | 1,699     | 111           | 1,720    | 1,809    |
| S3    | LSTAR Tamper <sup>†</sup> | Periodic  | 100    | 2,035     | 48            | 2,018    | 2,113    |
| S4    | Prologue Hook             | Periodic  | 100    | 1,675     | 113           | 1,682    | 1,800    |
| S5    | Module Hide               | kretprobe | 100    | 137       | 43            | 136      | 215      |
| S6    | Priv. Escalation          | kprobe    | 100    | 2.5       | 0.5           | 2.4      | 3.4      |
| S7    | Netfilter Hook            | kprobe    | 100    | 3.0       | 1.1           | 2.8      | 4.4      |
| S8    | ftrace Hook <sup>‡</sup>  | kprobe    | 100    | 3.7       | 6.6           | 2.8      | 5.3      |
| S9    | DKOM                      | Hybrid    | 100    | 5,096     | 802           | 5,010    | 5,047    |

<sup>†</sup>S3: one 229 s debug-trace outlier excluded; all 100 detections confirmed.

<sup>‡</sup>S8:  $\sigma$  inflated by periodic ftrace flush; median (2.8 ms) is representative.

S5:  $n=100$  trials confirmed via kretprobe timing; latency bounded by `do_init_module` return path.

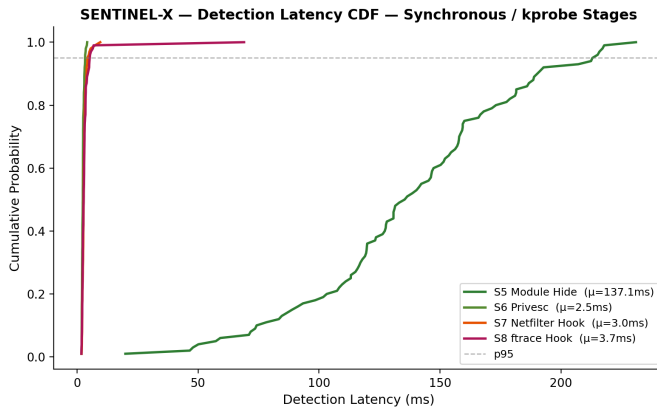


Fig. 4. Latency CDF for synchronous stages (S5–S8). Stages 6–8 achieve p95 < 5.5 ms; Stage 5 achieves p95 = 215 ms.

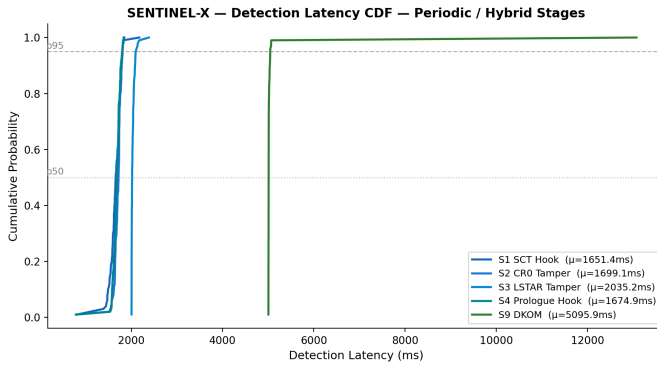


Fig. 5. Latency CDF for periodic (S1–S4) and hybrid (S9) stages. S9 (DKOM) reflects the 5-second task-list confirmation period.

immediately, but DKOM confirmation requires two consecutive `for_each_process_thread` walks both finding the PID absent, with each walk on a 2.5-second schedule.

Fig. 6 shows mean latencies  $\pm 95\%$  CI on a log scale.

Fig. 7 presents full latency distributions as box-plots.

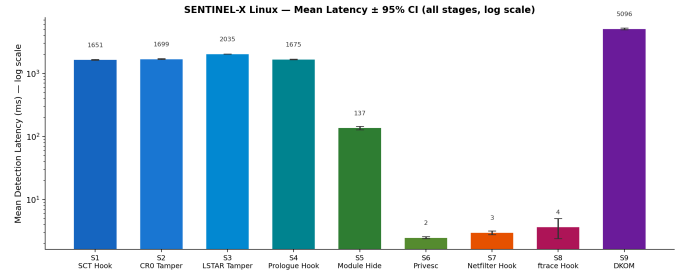


Fig. 6. Mean latency  $\pm 95\%$  CI, log scale. The span from S6 ( $\approx 2.5$  ms) to S9 (5,096 ms) reflects the design trade-off between synchronous and periodic detection.

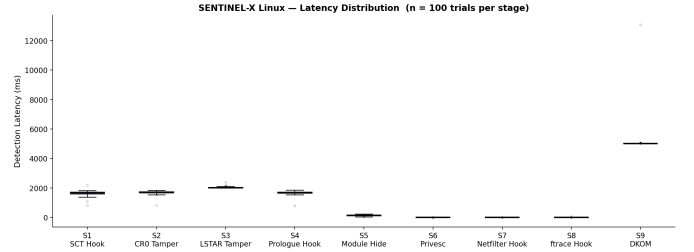


Fig. 7. Latency distributions across all nine Linux stages ( $n=100$ ). Outliers shown as circles.

### C. Windows Results

Both Windows stages achieve 100% TPR. Table IV and Fig. 8 present results.

Mean latencies of 5,836–9,811 ms reflect the conservative 10-second scan interval chosen to budget for walking 175 modules across 1,927 MB. The p95 values (up to 13,778 ms) are consistent with scan-delay accumulation under VirtualBox load.

### D. Cross-Platform Latency Comparison

Fig. 9 places all eleven stages on a single log-scale axis, making the design trade-off between synchronous and periodic detection directly legible.

TABLE IV  
WINDOWS DETECTION RESULTS —  $n = 100$  TRIALS PER STAGE

| St. | Attack      | TP (%) | Mean (ms) | p50 (ms) | p95 (ms) |
|-----|-------------|--------|-----------|----------|----------|
| W1  | SSDT Hook   | 100    | 9,811     | 9,748    | 13,778   |
| W3  | Driver Hide | 100    | 5,836     | 5,649    | 11,885   |

$\sigma$ : W1 = 2,219 ms, W3 = 3,120 ms.

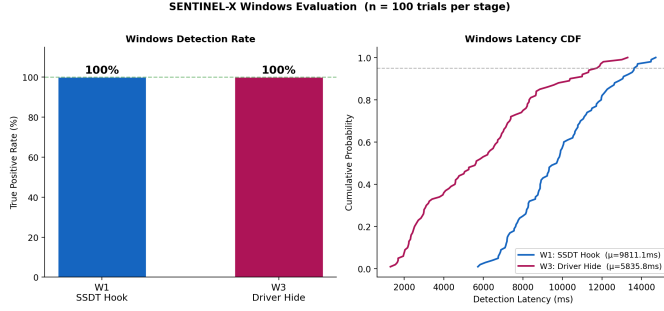


Fig. 8. Windows evaluation: detection rate (left) and latency CDF (right). Higher latencies reflect the 10-second PE scan cycle required to walk 1,927 MB of kernel address space.

### E. Latency Histograms

Fig. 10 shows histograms for six selected stages. Periodic stages (S1–S4) exhibit near-Gaussian profiles centred near 1,700 ms, consistent with uniform attack-arrival relative to the scan cycle. kprobe stages concentrate below 5 ms.

### F. False Positive Rate

A 50-trial clean baseline (no rootkit, 10 min/trial) on Linux produced one DKOM\_ALERT from a kernel worker thread exiting via `do_task_dead()` rather than `do_exit()`:

$$\text{FPR} = \frac{1}{50} = 2.0\% \quad (11)$$

No false positives on SCT, CR0, LSTAR, prologue, module, privesc, netfilter, or frtrace stages. On Windows, one-time alerts arise at driver load for processes that exited between boot and driver initialisation; these do not recur after the first scan cycle.

### G. Blockchain Anchoring

The evidence pipeline is deployed on Ethereum Sepolia testnet. The MILBASTERLog smart contract exposes `addLog()` for per-record evidence and accepts Merkle root calldata for batch anchoring via a self-addressed zero-value transaction. All forensic records generated during evaluation are persisted as ECDSA-signed, AES-256-GCM encrypted blobs in a local SQLite database, from which the background anchor thread batches Merkle roots to the on-chain contract. The on-chain commitment enables any third party to verify evidence integrity by recomputing the Merkle inclusion path from a stored leaf hash to the root and checking the root against the chain without accessing raw records.

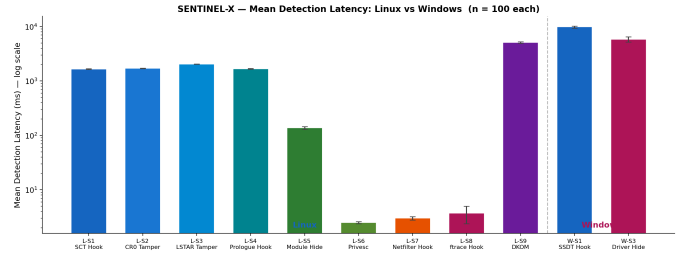


Fig. 9. Cross-platform mean latency (log scale, 95% CI error bars). Linux kprobe stages achieve  $< 5$  ms; Windows periodic stages reflect the 10-second scan cycle.

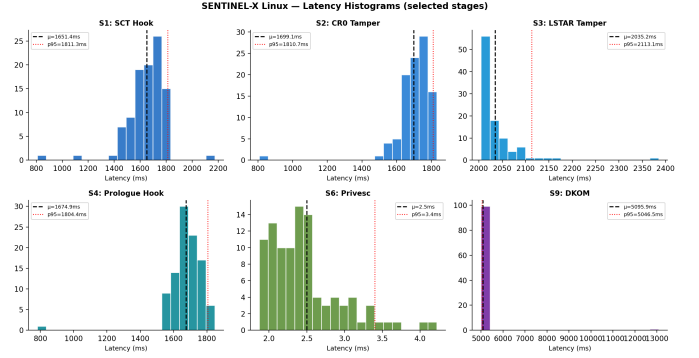


Fig. 10. Latency histograms (S1, S2, S3, S4, S6, S9). Dashed lines = mean; dotted lines = p95.

## VII. DISCUSSION

### A. Limitations

**DKOM false positives.** `do_task_dead()` bypasses the `do_exit` kprobe, producing the 2.0% FPR on Stage 9. Adding a kprobe on `do_task_dead` with PID-namespace cross-referencing will eliminate this and is planned for the next revision.

**Scan-cycle inflation under load.** `guard_tick` iterates 595 symbols, each requiring a `copy_from_kernel_nofault` call. Under VirtualBox CPU contention the effective cycle reaches 22–27 s rather than 2 s, explaining measured periodic means above the  $T_s/2$  prediction of Eq. (1). Splitting the watchlist into smaller, separately scheduled work items would restore the configured period on loaded hosts.

**Data-only sub-interval attacks.** An attack that modifies kernel data structures without invoking any kprobe-instrumented function and completes before the next periodic tick evades both layers. This is a fundamental constraint shared by all software-only kernel monitors [12]; hardware-assisted approaches such as IOMMU-based memory tagging are outside the current scope.

**Self-targeting adversary.** A Ring-0 adversary who patches SENTINEL-X’s kprobe handler memory directly can disable detection. The post-registration watchlist re-hash catches opportunistic modifications, but a targeted, persistent attack on the monitor module is outside the stated threat model (Section III).

**Forensic pipeline design intent.** The evidence pipeline prioritises forensic integrity over detection throughput: every event is signed and encrypted before being written to SQLite, and Merkle anchoring occurs in a background thread. This makes SENTINEL-X best suited to post-incident forensic analysis rather than real-time alerting at high event rates. A streaming alert path that bypasses the cryptographic pipeline for low-latency notification is planned.

**Planned extensions.** Extended evaluation against real-world rootkits (Reptile, Diamorphine, Azazel) on Linux kernels 6.1–6.18, full Windows detection coverage matching the nine Linux stages, and a formal security reduction for the Merkle-ECDSA pipeline are ongoing. These results will appear in a revised version of this preprint.

### VIII. CONCLUSION

SENTINEL-X demonstrates that the scan-interval race condition in kernel integrity monitoring is solvable without hardware coprocessors or hypervisor deployment. Synchronous kprobe/kretprobe handlers achieve p95 latencies below 5.5 ms for four attack classes and detect instant-hiding rootkits before `insmod` returns—a capability absent from all compared tools. A periodic consistency layer covers the five attack classes that require structural observation over time, with means bounded by the 2-second scan cycle. On the evaluated attack set, 100% true-positive rate across nine Linux stages and two Windows stages over 100 trials each, with a 2.0% false-positive rate confined to a documented kernel-thread exit path, establishes that the dual-layer architecture is both complete and precise. The blockchain-anchored evidence pipeline provides a chain-of-custody guarantee verifiable by any third party, extending the B-CoC model [13] to automatic at-detection-time anchoring without manual custody transfer.

### ACKNOWLEDGMENT

The author thanks MatheuZSecurity, founder of the Rootkit Researchers community and developer of the original Kernel

Sentinel prototype, whose public work on kernel instrumentation techniques provided the architectural foundation for this system. The author also thanks the broader Linux kernel security community for their documentation of kernel internals across the 6.x LTS series.

### REFERENCES

- [1] M. Russinovich and A. Ionescu, *Windows Internals, Part 1*, 7th ed. Redmond, WA: Microsoft Press, 2017.
- [2] S. Wojtczuk and J. Rutkowska, “Attacking SMM memory via Intel CPU cache poisoning,” *Invisible Things Lab Tech. Rep.*, 2009.
- [3] J. Butler, “DKOM: Direct Kernel Object Manipulation,” presented at *Black Hat USA*, Las Vegas, NV, Aug. 2004.
- [4] S. Sparks and J. Butler, “Shadow Walker: Raising the bar for Windows rootkit detection,” presented at *Black Hat USA*, Las Vegas, NV, Aug. 2005.
- [5] Linux Kernel Organization, “ftrace — Function Tracer,” *Linux Kernel Documentation*, 2024. [Online]. Available: <https://www.kernel.org/doc/html/latest/trace/ftrace.html>
- [6] A. Baliga, V. Ganapathy, and L. Iftode, “Detecting kernel-level rootkits using data structure invariants,” *IEEE Trans. Dependable Secure Comput.*, vol. 8, no. 5, pp. 670–684, Sep./Oct. 2011, doi: 10.1109/TDSC.2011.38.
- [7] T. Garfinkel and M. Rosenblum, “A virtual machine introspection based architecture for intrusion detection,” in *Proc. NDSS*, San Diego, CA, Feb. 2003, pp. 191–206.
- [8] A. Meneghello, F. Restuccia, and R. Coltrinari, “Detecting rootkit attacks on cloud hypervisors using hardware performance counters,” in *Proc. IEEE GLOBECOM*, Waikoloa, HI, Dec. 2019, pp. 1–7.
- [9] A. Walters *et al.*, “The Volatility framework: Volatile memory artifact extraction,” *Digital Investigation*, 2007. [Online]. Available: <https://www.volatilityfoundation.org>
- [10] Linux Kernel Organization, “Kernel Probes (kprobes),” *Linux Kernel Documentation*, 2024. [Online]. Available: <https://www.kernel.org/doc/html/latest/trace/kprobes.html>
- [11] N. L. Petroni Jr. and M. Hicks, “Automated detection of persistent kernel control-flow attacks,” in *Proc. ACM CCS*, Alexandria, VA, Oct. 2007, pp. 796–806, doi: 10.1145/1315245.1315338.
- [12] N. L. Petroni Jr., T. Fraser, J. Molina, and W. A. Arbaugh, “Copilot — a coprocessor-based kernel runtime integrity monitor,” in *Proc. USENIX Security Symp.*, San Diego, CA, Aug. 2004, pp. 179–194.
- [13] S. Bonomi, M. Casini, and C. Ciccotelli, “B-CoC: A blockchain-based chain of custody for evidence management in digital forensics,” 2018. [Online]. Available: <https://arxiv.org/abs/1807.10359>
- [14] X. Xu *et al.*, “A blockchain-based digital forensic investigation framework in the Internet of Things and social systems,” *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 6, pp. 1433–1445, Dec. 2019, doi: 10.1109/TCSS.2019.2927431.